# LD2-B Periodic Sparse Density Mesh Specification

Candidate-cell ownership, lifted components, winding fallback, canonical clipping, and crack-free rendering

mdstats development specification

2026-07-20

## Contents



## 1 Status and scope

This specification governs architecture gate **LD2-B** for `mdstats`. It begins from `mdstats 0.19.47a0`, where atomic local-sparse density fields already support exact HDR thresholds and logical-node clouds without dense materialization.

LD2-B adds triangular isosurface preparation and rendering for `PeriodicBlockScalarField3D`. It does not add framework sparse fields, automatic backend selection, multilevel AMR, mesh decimation, or GPU kernels.

## 2 Scientific invariants

LD2-B changes rendering only. It must not change:

1. CIC deposition;
2. `discrete_periodized_v1` scalar values;
3. field normalization;
4. scientific HDR thresholds;
5. logical-node coordinates;
6. periodic Cartesian metric;
7. source provenance.

The scientific threshold and the actual float32 contour level are recorded separately.

## 3 Logical node and cell model

The logical node lattice is

$$G_N = \mathbb{Z}_{N_1} \times \mathbb{Z}_{N_2} \times \mathbb{Z}_{N_3}.$$

A logical cell is identified by its lower periodic node index $\mathbf{c} \in G_N$. Its eight corners are

$$\mathbf{c} + \epsilon, \qquad \epsilon \in \{0, 1\}^3,$$

with node lookup modulo $\mathbf{N}$. A cell crosses contour level $\lambda$ when at least one corner is strictly above $\lambda$ and at least one corner is not strictly above it.

Candidate cells are generated only from stored positive nodes. Every logical cell adjacent to a stored node is considered, deduplicated by canonical C-order flat index, sorted, and evaluated through public periodic node access. This is complete for positive HDR levels because an all-implicit-zero cell cannot cross a positive level.

## 4 Candidate-cell components

Candidate cells are connected by six face-neighbor relations on the periodic cell lattice. Components are labeled in increasing canonical flat-cell order. Neighbor iteration order is fixed as

`-x, +x, -y, +y, -z, +z`

so labels and lifted charts are independent of hash insertion order.

## 5 Lifted charts and torus winding

For a periodic neighbor relation between canonical cells $\mathbf{c}$ and $\mathbf{d}$, the component stores a lifted integer coordinate $\tilde{\mathbf{c}} \in \mathbb{Z}^3$. Crossing a periodic boundary changes the lifted coordinate by the corresponding unwrapped unit step.

Breadth-first lifting assigns one lifted coordinate to every component cell. If a previously assigned cell is reached with a different lifted coordinate, the cycle carries nonzero torus winding. The component record stores all deterministic residual winding vectors.

A nonwinding component is meshed in one compact chart. A winding component uses the following deterministic fallback:

1. dense canonical fallback if the logical node count does not exceed `max_dense_fallback_nodes`;
2. otherwise logical-node cloud fallback if `allow_cloud_fallback=True`;
3. otherwise raise `GraphComplexityError` before mesh allocation.

# 6 Cell-aware Lewiner contouring

The reference production algorithm contours every owned candidate cell exactly once using the Lewiner implementation supplied by `skimage.measure.marching_cubes` on a $2 \times 2 \times 2$ cell volume. This preserves source-cell identity without post hoc ownership inference from a black-box component volume.

For each candidate cell:

1. gather its eight scalar values in lifted corner order;
2. convert them to contiguous `float32`;
3. use the component render level;
4. run Lewiner marching cubes with spacing $(1/N_1, 1/N_2, 1/N_3)$;
5. add the lifted lower-cell coordinate divided by $\mathbf{N}$;
6. append triangles in canonical component-cell order.

Calling Lewiner once per owned cell is the correctness path for LD2-B. Later gates may batch cells only if they preserve exact ownership, determinism, and geometry.

The method is based on Lorensen and Cline's marching-cubes construction and Lewiner et al.'s topologically consistent case resolution.

# 7 Contour-level policy

Let $\lambda_{\mathrm{HDR}}$ be the scientific HDR threshold. The contour library operates in `float32`. Define the component render level by:

1. cast $\lambda_{\mathrm{HDR}}$ to `float32`;
2. if it is not strictly inside the component's float32 scalar range, move it to the nearest representable interior value;
3. if a cell still has no surface, it contributes no triangles;
4. do not globally lower the contour merely because one owned cell is flat.

The result records both $\lambda_{\mathrm{HDR}}$ and the actual render level.

# 8 Canonical replication and clipping

A lifted nonwinding component may cross a canonical boundary. Its unwrapped triangles are translated by every integer image shift whose bounding box can intersect the canonical cube $[0, 1]^3$. Each translated triangle is clipped against the six canonical fractional half-spaces using deterministic Sutherland-Hodgman polygon clipping. Clipped polygons are fan-triangulated while preserving orientation.

Independent vertex wrapping is forbidden.

Vertices are canonicalized after clipping with tolerance

$$\tau_x = 10^{-10} L_{\mathrm{ref}}, \qquad L_{\mathrm{ref}} = \max_i \|\mathbf{a}_i\|_2.$$

Canonicalization uses quantized Cartesian keys plus exact distance confirmation. Faces are canonicalized by cyclic orientation-preserving rotation for output and by sorted vertex triplets for duplicate detection. Degenerate faces are removed.

# 9 Mesh diagnostics

The prepared mesh records:

```
PeriodicSparseDensityMesh3D(
    vertices_fractional,
    vertices_cartesian,
    faces,
```

```
    scientific_hdr,
    render_level,
    resources,
    topology,
    provenance,
)
```

Required topology diagnostics include:

- candidate-cell count;
- component count;
- winding-component count;
- residual winding vectors;
- duplicate-face count removed;
- degenerate-face count removed;
- interior edge incidence failures;
- canonical-boundary cut-edge count;
- opposite-boundary paired-edge count;
- unpaired boundary-edge count;
- maximum Cartesian mesh-edge length;
- fallback mode.

# 10 Periodic seam validation

After canonical clipping, ordinary Euclidean watertightness is not required. Validation distinguishes:

1. interior edges, which must have incidence two;
2. canonical-boundary edges, which may have incidence one;
3. boundary edges, which must pair across the corresponding opposite face after a unit fractional translation within $10^{-10} L_{\mathrm{ref}}$.

Every mesh edge must satisfy

$$\ell_{\mathrm{edge}} \le (\|\mathbf{b}_1\| + \|\mathbf{b}_2\| + \|\mathbf{b}_3\|)\,(1 + 10^{-10}),$$

where $\mathbf{b}_i = \mathbf{a}_i / N_i$.

# 11 Resource policy

Before triangle storage, LD2-B bounds:

- candidate cells;
- cell-value workspace;
- raw per-cell vertices and faces;
- canonical replication count;
- clipped vertices and faces;
- dense-fallback nodes;
- Plotly replicated faces and traces.

No mesh is silently decimated. Limits are explicit and failures occur before the corresponding large allocation.

# 12 Rendering integration

For `render_mode="mesh"`:

- dense fields retain the existing dense mesher;
- local-sparse fields use LD2-B preparation;
- canonical meshes render once;

- `display_replication="match_graph"` translates the prepared canonical mesh into every primary graph image without recomputing density or contouring;
- winding fields follow the recorded dense or node-cloud fallback;
- trace provenance records field identity, scientific threshold, render level, component counts, image shift, and fallback mode.

# 13 Public API

The stage exports:

```
identify_sparse_mesh_candidate_cells(...)
label_periodic_cell_components(...)
prepare_sparse_density_mesh(...)
validate_periodic_canonical_mesh(...)
```

and immutable records for candidate cells, lifted components, resource accounting, topology diagnostics, and the prepared mesh.

# 14 Acceptance tests

LD2-B passes only if:

```
interior mesh-edge incidence failures = 0
unpaired non-boundary edge count = 0
opposite-boundary seam mismatch <= 1e-10 * L_ref
duplicate canonical face count = 0
maximum mesh edge <= cell-diagonal upper bound * (1 + 1e-10)
```

The required fixtures are:

1. one orthogonal interior Gaussian;
2. one skewed LTA-primitive Gaussian;
3. face-crossing cloud;
4. edge-crossing cloud;
5. corner-crossing cloud;
6. two overlapping clouds;
7. separated components;
8. partial terminal blocks;
9. block-order permutation;
10. a nonzero-winding synthetic shell;
11. deterministic dense fallback;
12. deterministic node-cloud fallback;
13. expanded-cell mesh replication;
14. resource-limit failure before large allocation.

Sparse and dense canonical meshes are compared by canonicalized vertex/face sets within the stated geometry tolerance, not by raw library vertex ordering.

# 15 Non-objectives

LD2-B does not implement:

- framework sparse density preparation;
- automatic backend selection;
- multilevel meshes;
- adaptive decimation;
- tiled sparse contouring for percolating torus components;
- GPU rendering.

# 16 References


1. Lorensen, W. E., and H. E. Cline. "Marching Cubes: A High Resolution 3D Surface Construction Algorithm." *ACM SIGGRAPH Computer Graphics* **21** (1987): 163-169. DOI: 10.1145/37402.37422.
2. Lewiner, T., H. Lopes, A. W. Vieira, and G. Tavares. "Efficient Implementation of Marching Cubes' Cases with Topological Guarantees." *Journal of Graphics Tools* **8** (2003): 1-15. DOI: 10.1080/10867651.2003.10487582.
3. Berger, M. J., and P. Colella. "Local Adaptive Mesh Refinement for Shock Hydrodynamics." *Journal of Computational Physics* **82** (1989): 64-84. DOI: 10.1016/0021-9991(89)90035-1.


# 17 Revision status after `0.19.75a0`

Revision Stage 2 separates sparse-mesh face-count semantics through `DensityMeshFaceContract`:

- `raw_extraction_face_limit` is a runtime-capped computational safety limit;
- `visual_target_faces` is a soft scene-fitting target;
- `standalone_final_face_limit` is an optional terminal limit for calls without a scene controller.

`prepare_sparse_density_mesh(..., face_contract=...)` is the normative API. The legacy `max_faces` and `max_raw_faces` arguments remain a standalone compatibility path and may not be mixed with the explicit contract. A scene-controller target miss is returned as `DensityMeshFaceReport` metadata and is not rejected before the Stage-3 fitting controller. The historical 250,000 limit remains the standalone default only.